

OptFSST: Optimized FSST String Compression

Hedi Chehaidar*, Mihail Stoian, Moritz Stargalla, Andreas Kipf

[Data Systems, Applied Discrete Mathematics] Lab @UTN

*did the heavy lifting (2x IOI)

@ADMS'26, August 31, 2026

Motivation: File Formats x String Encodings



BtrBlocks

Motivation: File Formats x String Encodings

FSST: Fast Random Access String Compression

Peter Boncz
CWI
boncz@cwi.nl

Thomas Neumann
TUM
neumann@in.tum.de

Viktor Leis
FSU Jena
viktor.leis@uni-jena.de

ABSTRACT

Strings are prevalent in real-world data sets. They often occupy a large fraction of the data and are slow to process. In this work, we present Fast Static Symbol Table (FSST), a *lightweight* compression scheme for strings. On text data, FSST offers decompression and compression speed similar to or better than the best speed-optimized compression methods, such as LZ4, yet offers significantly better compression factors. Moreover, its use of a static symbol table allows *random access* to individual, compressed strings, enabling lazy decompression and query processing on compressed data. We believe these features will make FSST a valuable piece in the standard compression toolbox.

PVLDB Reference Format:

Peter Boncz, Thomas Neumann, and Viktor Leis. FSST: Fast Random Access String Compression. PVLDB 16(11): 2016-2021.

corpus
(uncompressed)

```
http://in.tum.de
http://cwi.nl
www.uni-jena.de
www.wikipedia.org
http://www.vldb.org
...
```

symbol table

0	http://	7
1	www.	4
2	uni-jena	8
3	.de	3
4	.org	4
5	a	1
6	in.tum	6
7	cwi.nl	6
8	wikipedi	8
9	vldb	4
...		
255		

symbol length

corpus
(compressed)

```
063
07
123
1854
0194
...
```

Motivation: FSST

- All *newer* file formats switched to FSST as their main string encoding.
 - (DuckDB switched to DICT_FSST.)
- FSST uses a *greedy* encoder: match the longest matching symbol.

Motivation: FSST

- All *newer* file formats switched to FSST as their main string encoding.
 - (DuckDB switched to DICT_FSST.)
- FSST uses a *greedy* encoder: match the longest matching symbol.

Symbol table:

admsi	adms	isawesome	s	awesome
--------------	-------------	------------------	----------	----------------

FSST(greedy):

a	d	m	s	i	s	a	w	e	s	o	m	e
---	---	---	---	---	---	---	---	---	---	---	---	---

Motivation: FSST

- All *newer* file formats switched to FSST as their main string encoding.
 - (DuckDB switched to DICT_FSST.)
- FSST uses a *greedy* encoder: match the longest matching symbol.
- Is there a better way? 🤔

Motivation: FSST

- All *newer* file formats switched to FSST as their main string encoding.
 - (DuckDB switched to DICT_FSST.)
- FSST uses a *greedy* encoder: match the longest matching symbol.
- Is there a better way? 🤔

Symbol table:

admsi	adms	isawesome	s	awesome
--------------	-------------	------------------	----------	----------------

FSST(greedy):

a	d	m	s	i	s	a	w	e	s	o	m	e
---	---	---	---	---	---	---	---	---	---	---	---	---



OptFSST(DP):

a	d	m	s	i	s	a	w	e	s	o	m	e
---	---	---	---	---	---	---	---	---	---	---	---	---



Motivation: FSST

- All *newer* file formats switched to FSST as their main string encoding.
 - (DuckDB switched to DICT_FSST.)
- FSST uses a *greedy* encoder: match the longest matching symbol.
- It's not clear how much compression performance is left on the table.
- Especially in the case of FSST:

better compression $\Rightarrow$ **faster** decoding $\Rightarrow$ **faster** queries,

- due to fewer symbols.

Motivation: FSST

- All *newer* file formats switched to FSST as their main string encoding.
 - (DuckDB switched to DICT_FSST.)
- FSST uses a *greedy* encoder: match the longest matching symbol.
- It's not clear how much compression performance is left on the table.
- Especially in the case of FSST:

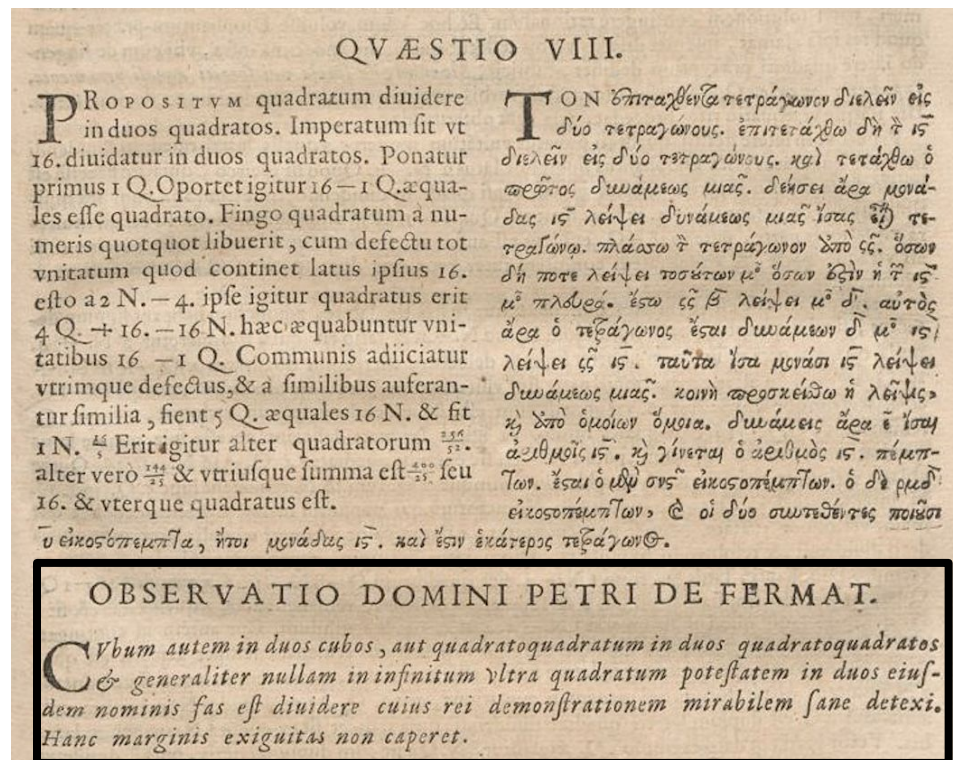
better compression $\Rightarrow$ **faster** decoding $\Rightarrow$ **faster** queries,

- due to fewer symbols.

¹ We experimented with an encoding function that frames string compression as a dynamic programming problem, rather than applying `findLongestSymbol()` greedily. However, we found that the compression factor only marginally improves, while encoding performance is severely affected.

Motivation: FSST

- How it felt to me:



OptFSST: Results

- Since I missed the numbers, here they are.
- Improvement over vanilla FSST:

	avg.	max.
OptFSST	7.3%	47.7%
OptFSST12	17.0%	91.5%

- OptFSST12 improves FSST12's decompression throughput by **1.2x** (avg).

FSST: Tutorial

Why?

- String data dominating (see Amazon's fleet analysis).
- Random access.
- Fast decompression.
- Compressed execution.

FSST: Tutorial

How?

<i>corpus</i> <i>(uncompressed)</i>	<i>symbol table</i>	<i>corpus</i> <i>(compressed)</i>
http://in.tum.de	0 http:// 7	063
http://cwi.nl	1 www. 4	07
www.uni-jena.de	2 uni-jena 8	123
www.wikipedia.org	3 .de 3	1854
http://www.vldb.org	4 .org 4	0194
...	5 a 1	...
	6 in.tum 6	
	7 cwi.nl 6	
	8 wiki pedi 8	
	9 vldb 4	
	...	
	255	
	<i>symbol length</i>	

FSST: Escaping

- Codes mapping to symbols: 0-254 & code 255 = escape code.
- Compression:
 - No symbol matching current prefix
⇒ place the escape code followed by raw byte.
- Decompression:
 - Escape code encountered
⇒ place next byte as is and advance by 2.

FSST: Symbol Table Construction

- 5 iterations over a sample (generations).
- Compress the sample and keep counts (`compressCount`).
- Select symbols for the next table based on static gain (`makeTable`).
- Keep best table.

FSST: Optimality Issues

- Greedy algorithm for *table construction* and *full column encoding*.
- Frequencies only for single symbols and symbol pairs (context length = 2).
- Conflicting symbols selected (overlapping).

OptFSST: Our Ingredients

- Dynamic programming for:
 - table construction.
 - text encoding.

(What's cool: You can use the DP *just* for the table construction and be better.)

- Multi-dimensional symbol counter (in this paper: 3D).

(LLMs also need longer context sizes.)

OptFSST: Disclaimer

- This is still not the *optimal* way of encoding the text.
- We show that it's NP-hard to find the optimal symbol table
 - (When the alphabet is part of the input.)
 - Latest proofs on tokenization's NP-hardness don't fix the token length.
 - In FSST: 8 byte.

OptFSST: Disclaimer

- This is still not the *optimal* way of encoding the text.
- We show that it's NP-hard to find the optimal symbol table
 - (When the alphabet is part of the input.)
 - Latest proofs on tokenization's NP-hardness don't fix the token length.
 - In FSST: 8 byte.
- But – we're optimal if you already optimized for a symbol table (due to the DP).
 - *Empirically* doing DP also for the symbol table helps.

OptFSST: Disclaimer

- This is still not the *optimal* way of encoding the text.
- We show that it's NP-hard to find the optimal symbol table
 - (When the alphabet is part of the input.)
 - Latest proofs on tokenization's NP-hardness don't fix the token length.
 - In FSST: 8 byte.
- But – we're optimal if you already optimized for a symbol table (due to the DP).
 - *Empirically* doing DP also for the symbol table helps.
- AI Story (April 29th – on the *driverless* Nuremberg's metro).

OptFSST: Disclaimer

- This is still not the *optimal* way of encoding the text.
- We show that it's NP-hard to find the optimal symbol table
 - (When the alphabet is part of the input.)
 - Latest proofs on tokenization's NP-hardness don't fix the token length.
 - In FSST: 8 byte.
- But – we're optimal if you already optimized for a symbol table (due to the DP).
 - *Empirically* doing DP also for the symbol table helps.
- AI Story (April 29th – on the *driverless* Nuremberg's metro):
 - ChatGPT5.5 Pro for 2-byte symbols.
 - Gemini found issue in the proof.
 - ChatGPT fixed it with the feedback (Gemini approved).
 - Asked to generalize to 8-byte symbols → was beyond my expertise.
 - Moritz (our math guy) completed the proof.

Contributions: Dynamic Programming

$dp[i]$ = smallest compressed size of the first i bytes of the given text.

$$dp[0] = 0$$

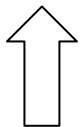
$\Rightarrow$ result for the whole text: $dp[n]$.

Contributions: Dynamic Programming

$$dp[i] = \min(2 + dp[i - 1], 1 + \min_{\substack{0 \leq j < i \\ s[j:i] \in \text{symbols}}} dp[j])$$

Contributions: Dynamic Programming

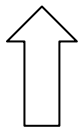
$$dp[i] = \min(2 + dp[i - 1], 1 + \min_{\substack{0 \leq j < i \\ s[j:i] \in \text{symbols}}} dp[j])$$



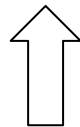
Escape last byte

Contributions: Dynamic Programming

$$dp[i] = \min(2 + dp[i - 1], 1 + \min_{\substack{0 \leq j < i \\ s[j:i] \in \text{symbols}}} dp[j])$$



Escape last byte



Last byte belongs to a symbol

Greedy Encoding: Compression Example

Available symbols:

a	aa	ac	aac	cb
---	----	----	-----	----

Text to compress:

a	c	b	a	a	c	b	a	a
---	---	---	---	---	---	---	---	---

Greedy:

a	c	b	a	a	c	b	a	a
---	---	---	---	---	---	---	---	---

Greedy Encoding: Compression Example

Available symbols:

a	aa	ac	aac	cb
---	----	----	-----	----

Text to compress:

a	c	b	a	a	c	b	a	a
---	---	---	---	---	---	---	---	---

Greedy:

a	c	b	a	a	c	b	a	a
---	---	---	---	---	---	---	---	---

escaped $\Rightarrow$ 2 bytes to compressed size!

Greedy Encoding: Compression Example

Available symbols:

a	aa	ac	aac	cb
---	----	----	------------	----

Text to compress:

a	c	b	a	a	c	b	a	a
----------	----------	----------	----------	----------	----------	----------	----------	----------

Greedy:

a	c	b	a	a	c	b	a	a
----------	----------	----------	----------	----------	----------	----------	----------	----------

Greedy Encoding: Compression Example

Available symbols:

a	aa	ac	aac	cb
---	----	----	-----	----

Text to compress:

a	c	b	a	a	c	b	a	a
---	---	---	---	---	---	---	---	---

Greedy:

a	c	b	a	a	c	b	a	a
---	---	---	---	---	---	---	---	---

escaped $\Rightarrow$ 2 bytes to compressed size!

Greedy Encoding: Compression Example

Available symbols:

a	aa	ac	aac	cb
---	----	----	-----	----

Text to compress:

a	c	b	a	a	c	b	a	a
---	---	---	---	---	---	---	---	---

Greedy:

a	c	b	a	a	c	b	a	a
---	---	---	---	---	---	---	---	---

Total compressed size = **8 byte**.

DP Encoding: Compression Example

Available symbols:

a	aa	ac	aac	cb
----------	----	----	-----	----

Text to compress:

a	c	b	a	a	c	b	a	a
----------	----------	----------	----------	----------	----------	----------	----------	----------

DP:

a	c	b	a	a	c	b	a	a
----------	----------	----------	----------	----------	----------	----------	----------	----------

0	1							
----------	----------	--	--	--	--	--	--	--

DP Encoding: Compression Example

Available symbols:

a	aa	ac	aac	cb
---	----	----	-----	----

Text to compress:

a	c	b	a	a	c	b	a	a
---	---	---	---	---	---	---	---	---

DP:

a	c	b	a	a	c	b	a	a
---	---	---	---	---	---	---	---	---

0	1	1						
---	---	---	--	--	--	--	--	--

DP Encoding: Compression Example

Available symbols:

a	aa	ac	aac	cb
---	----	----	-----	----

Text to compress:

a	c	b	a	a	c	b	a	a
---	---	---	---	---	---	---	---	---

DP:

a	c	b	a	a	c	b	a	a
---	---	---	---	---	---	---	---	---

0	1	1	2					
---	---	---	---	--	--	--	--	--

DP Encoding: Compression Example

Available symbols:

a	aa	ac	aac	cb
----------	----	----	-----	----

Text to compress:

a	c	b	a	a	c	b	a	a
----------	----------	----------	----------	----------	----------	----------	----------	----------

DP:

a	c	b	a	a	c	b	a	a
----------	----------	----------	----------	----------	----------	----------	----------	----------

0	1	1	2	3				
----------	----------	----------	----------	----------	--	--	--	--

DP Encoding: Compression Example

Available symbols:

a	aa	ac	aac	cb
---	----	----	-----	----

Text to compress:

a	c	b	a	a	c	b	a	a
---	---	---	---	---	---	---	---	---

DP:

a	c	b	a	a	c	b	a	a
---	---	---	---	---	---	---	---	---

0	1	1	2	3	3				
---	---	---	---	---	---	--	--	--	--

DP Encoding: Compression Example

Available symbols:

a	aa	ac	aac	cb
---	----	----	-----	----

Text to compress:

a	c	b	a	a	c	b	a	a
---	---	---	---	---	---	---	---	---

DP:

a	c	b	a	a	c	b	a	a
---	---	---	---	---	---	---	---	---

0	1	1	2	3	3	3			
---	---	---	---	---	---	---	--	--	--

DP Encoding: Compression Example

Available symbols:

a	aa	ac	aac	cb
---	----	----	-----	----

Text to compress:

a	c	b	a	a	c	b	a	a
---	---	---	---	---	---	---	---	---

DP:

a	c	b	a	a	c	b	a	a
---	---	---	---	---	---	---	---	---

0	1	1	2	3	3	3	4		
---	---	---	---	---	---	---	---	--	--

DP Encoding: Compression Example

Available symbols:

a	aa	ac	aac	cb
---	----	----	-----	----

Text to compress:

a	c	b	a	a	c	b	a	a
---	---	---	---	---	---	---	---	---

DP:

a	c	b	a	a	c	b	a	a
---	---	---	---	---	---	---	---	---

0	1	1	2	3	3	3	4	5	
---	---	---	---	---	---	---	---	---	--

DP Encoding: Compression Example

Available symbols:

a	aa	ac	aac	cb
---	----	----	-----	----

Text to compress:

a	c	b	a	a	c	b	a	a
---	---	---	---	---	---	---	---	---

DP:

a	c	b	a	a	c	b	a	a
---	---	---	---	---	---	---	---	---

0	1	1	2	3	3	3	4	5	5
---	---	---	---	---	---	---	---	---	---

DP Encoding: Compression Example

Available symbols:

a	aa	ac	aac	cb
---	----	----	-----	----

Text to compress:

a	c	b	a	a	c	b	a	a
---	---	---	---	---	---	---	---	---

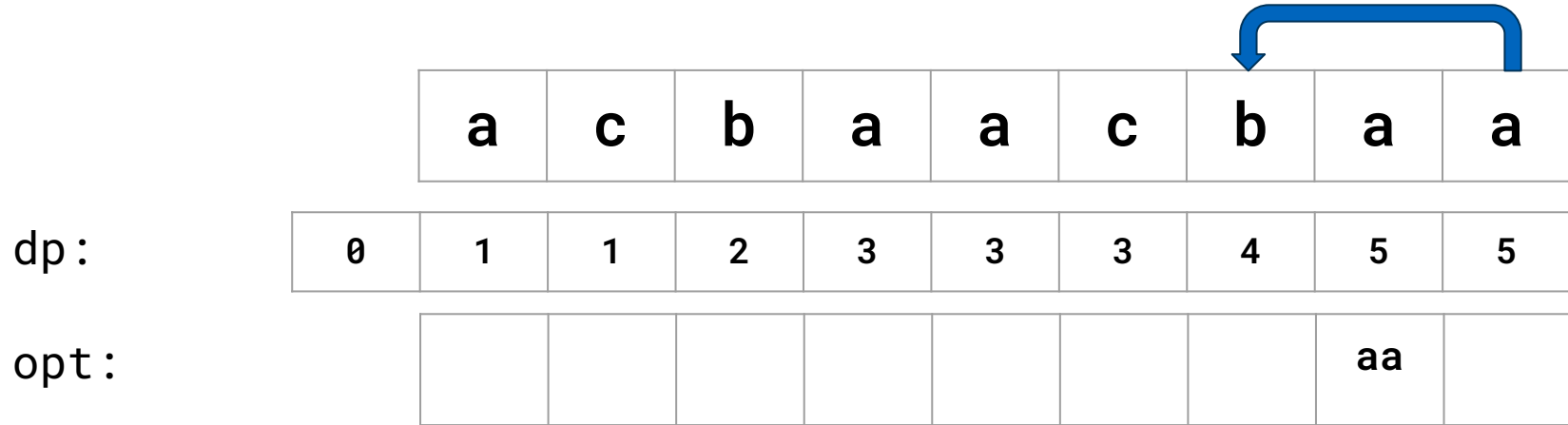
DP:

a	c	b	a	a	c	b	a	a
---	---	---	---	---	---	---	---	---

0	1	1	2	3	3	3	4	5	5
---	---	---	---	---	---	---	---	---	---

⇒ Total compressed size = **5 byte**.

DP Encoding: Retrieving the Symbols



DP Encoding: Retrieving the Symbols

									
	a	c	b	a	a	c	b	a	a
dp:	0	1	1	2	3	3	3	4	5
opt:						cb		aa	

DP Encoding: Retrieving the Symbols



	a	c	b	a	a	c	b	a	a	
dp:	0	1	1	2	3	3	3	4	5	5
opt:				aa		cb		aa		

DP Encoding: Retrieving the Symbols

										
	a	c	b	a	a	c	b	a	a	
dp:	0	1	1	2	3	3	3	4	5	5
opt:		cb		aa		cb		aa		

DP Encoding: Retrieving the Symbols



a	c	b	a	a	c	b	a	a
----------	----------	----------	----------	----------	----------	----------	----------	----------

dp:

0	1	1	2	3	3	3	4	5	5
---	---	---	---	---	---	---	---	---	---

opt:

a	cb		aa		cb		aa	
----------	-----------	--	-----------	--	-----------	--	-----------	--

DP Encoding: Issues

- This is going to be slow if we're not optimizing it..

DP Encoding: Issues

- This is going to be slow if we're not optimizing it..
- We did two major changes:
 1. Single-pass *backwards* DP.
 2. Highly optimized trie to match symbols (loop unrolling, branch prediction).

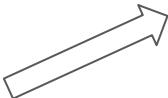
DP Encoding: Fixes

- This is going to be slow if we're not optimizing it..
- We did two major changes:
 1. Single-pass *backwards* DP.
 2. Highly optimized trie to match symbols (loop unrolling, branch prediction).

$$dp[i] = \min(2 + dp[i + 1], 1 + \min_{\substack{i < j \leq n \\ s[i:j] \in symbols}} dp[j])$$

DP Encoding: Fixes

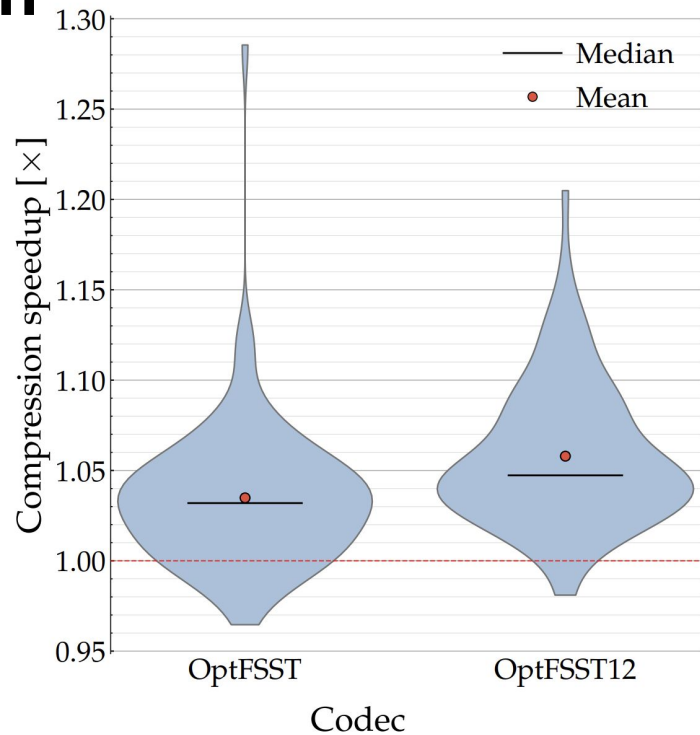
- This is going to be slow if we're not optimizing it..
- We did two major changes:
 1. Single-pass *backwards* DP.
 2. Highly optimized trie to match symbols (loop unrolling, branch prediction).

$$dp[i] = \min(2 + dp[i + 1], 1 + \min_{\substack{i < j \leq n \\ s[i:j] \in \text{symbols}}} dp[j])$$


where the trie is needed.

DP Encoding: Symbol Search

- Symbol search via a trie.
- Compression time speedup with *loop unrolling* and *branch prediction compiler hints*.



Contributions: Multi-Dimensional Counter (3D)

- Current FSST only counts frequencies of single symbols & symbol pairs.

⇒ OptFSST also track frequencies of *symbol triples* via a hashmap.

(Constant-time operation.)

Contributions: Pruning

- Gains remain static through table filling and do not account for current selected symbols.

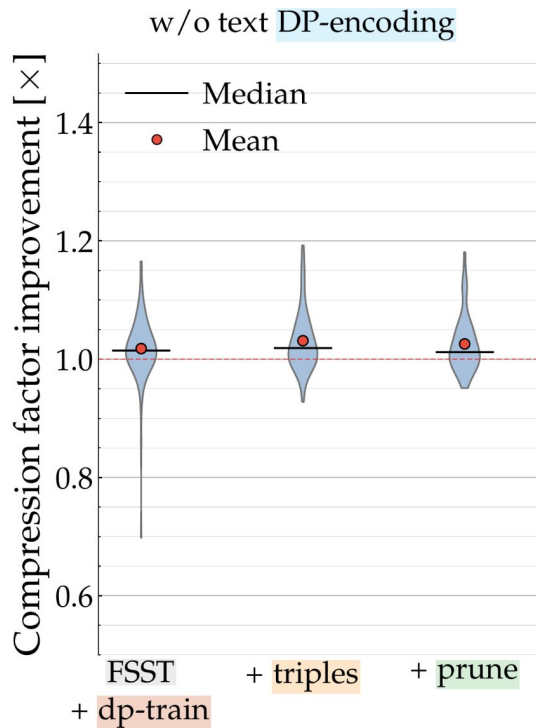
⇒ We *dynamically* adapt the gains during compression.

(More details in the paper.)

Benchmarks: Datasets

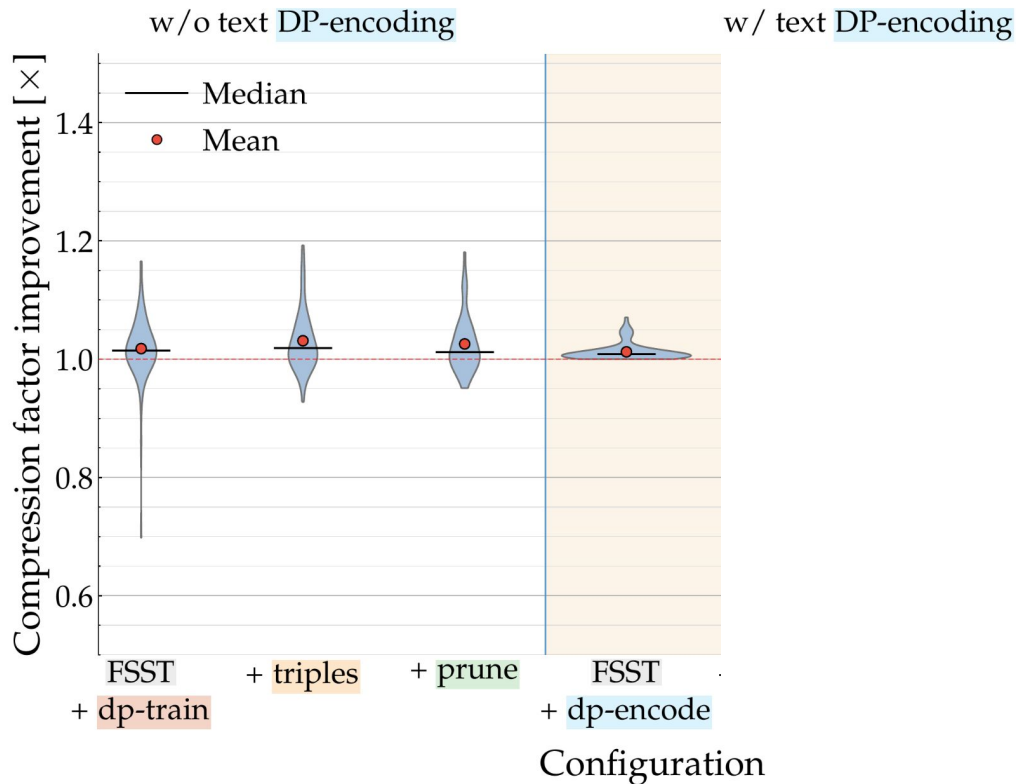
- 92 columns:
 - dbtext, NextiaJD, CyclicJoinBench, Clickbench, and PublicBIbenchmark.
- Tables reduced to first **100K** rows.
- String columns having at least **1K** rows and size $\leq$ **35MB**.
- **FSST** compressed size $\leq$ Dictionary encoding size.

OptFSST: Compression Improvement

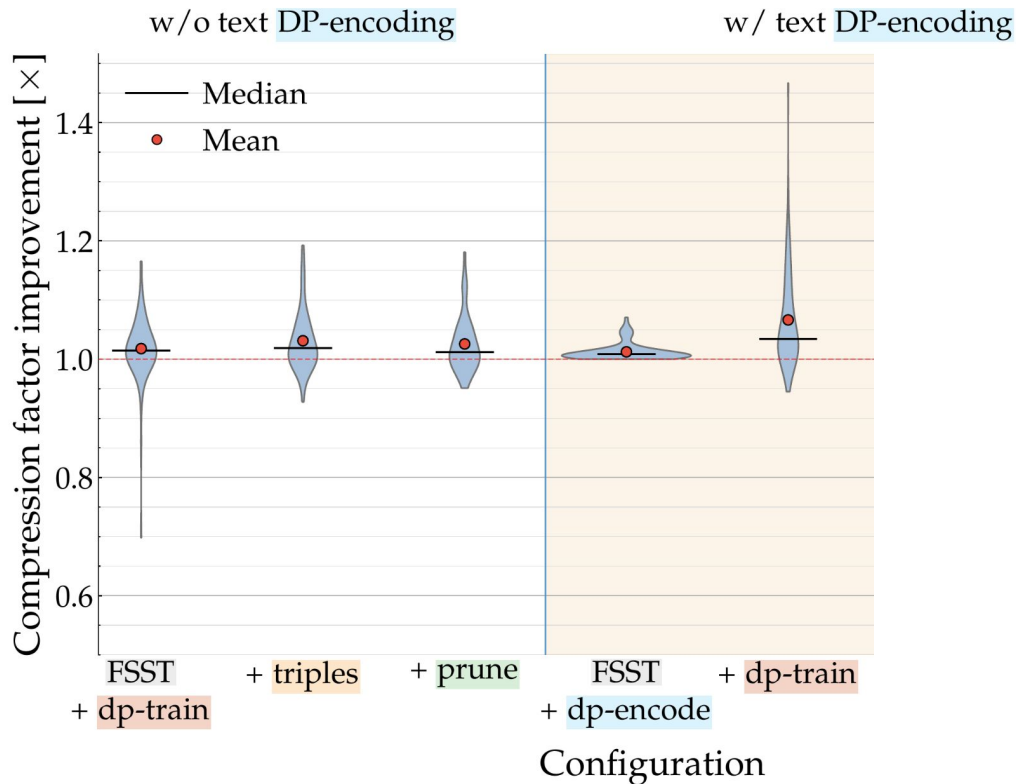


Configuration

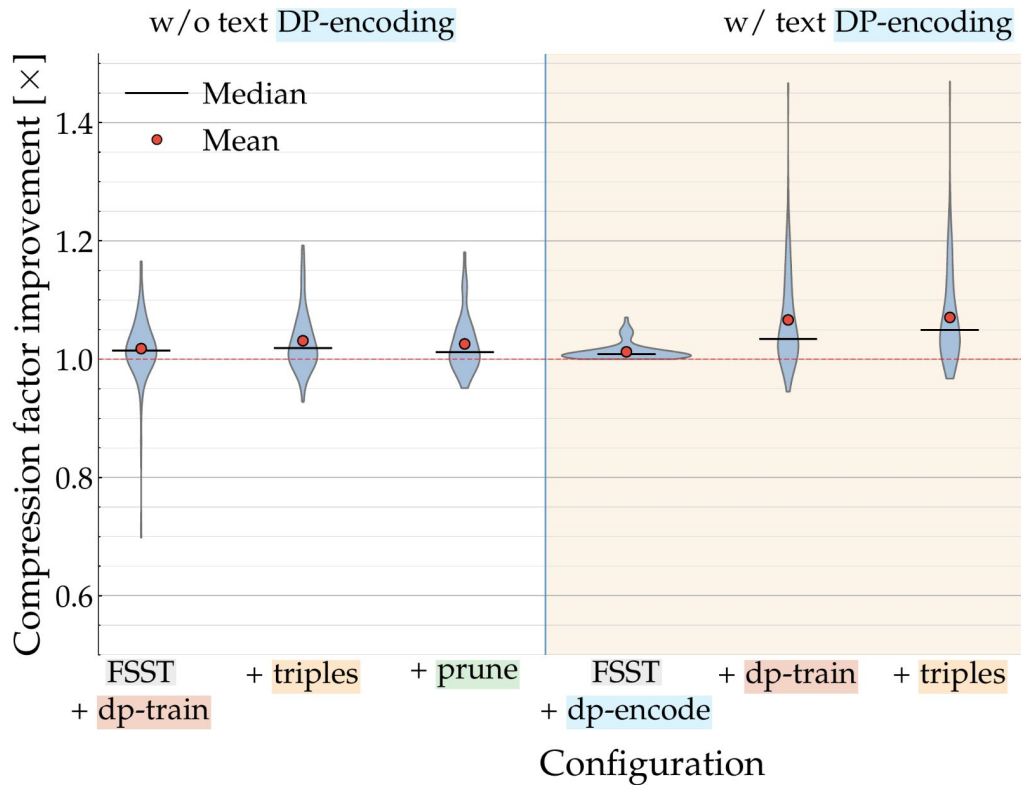
OptFSST: Compression Improvement



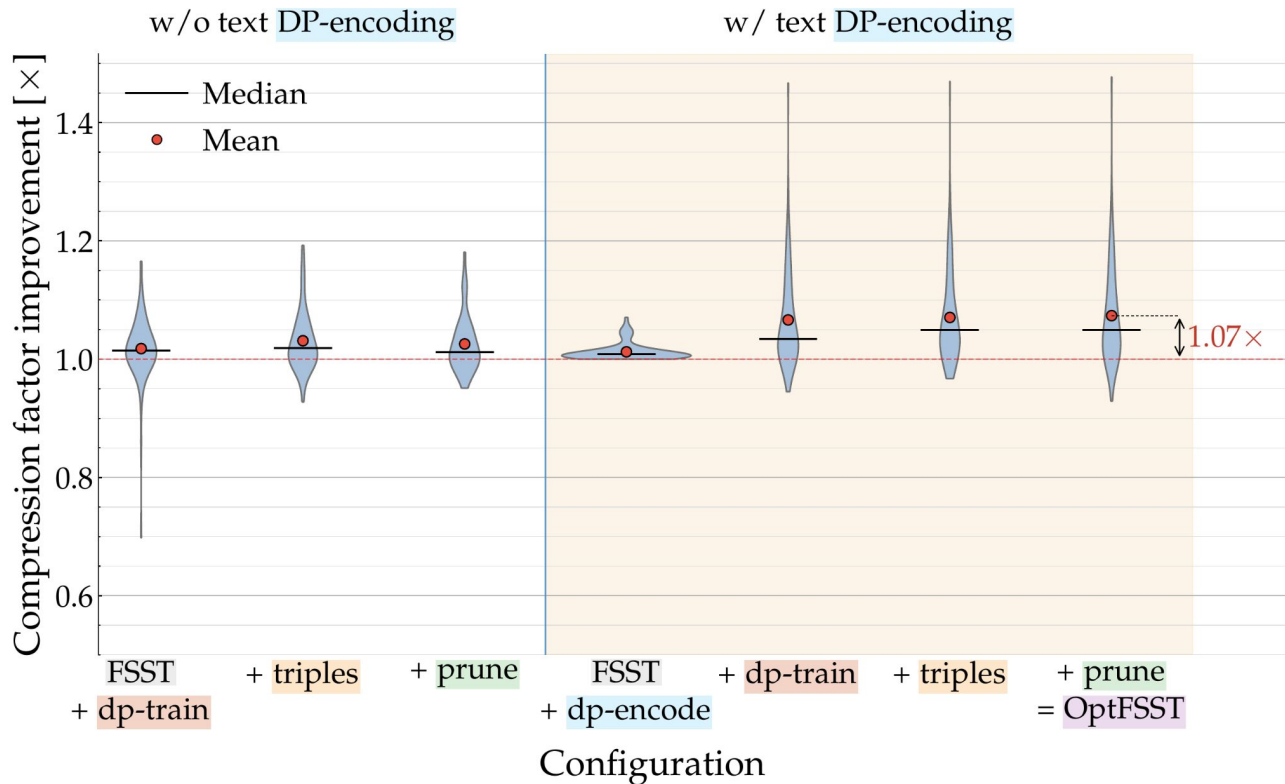
OptFSST: Compression Improvement



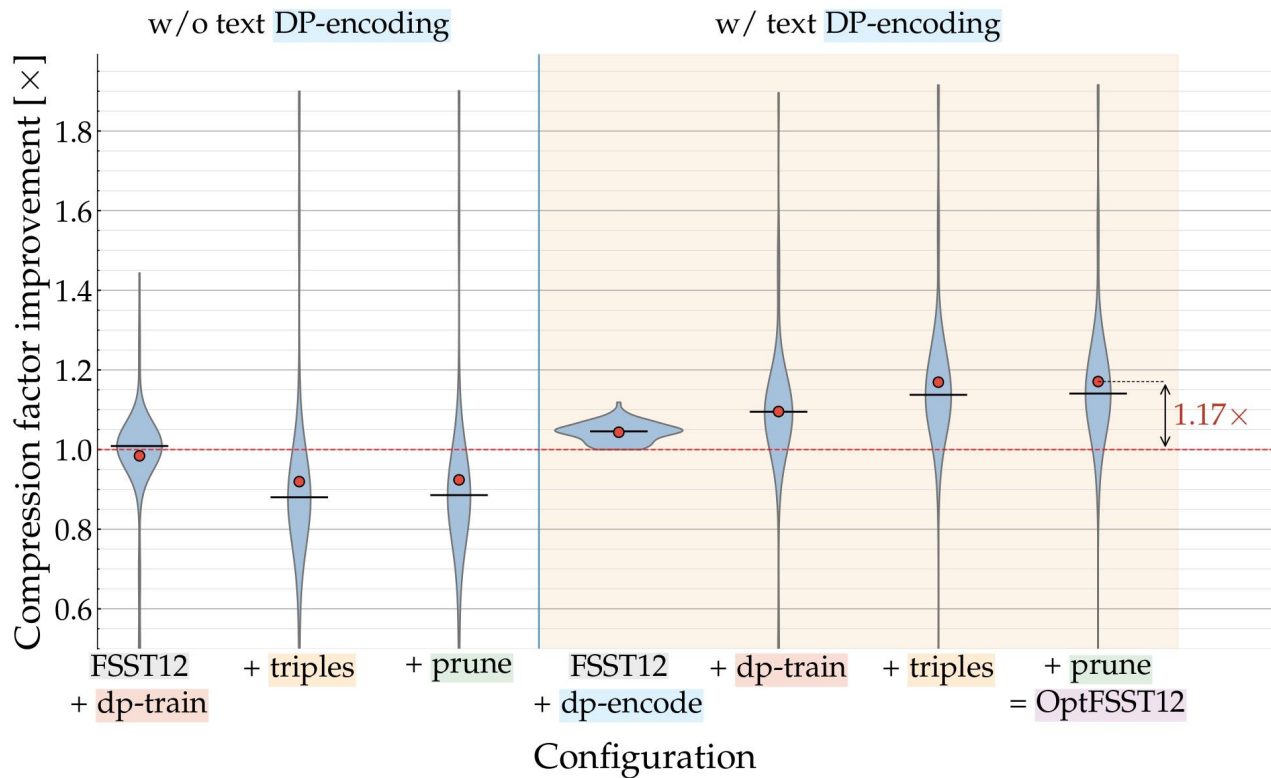
OptFSST: Compression Improvement



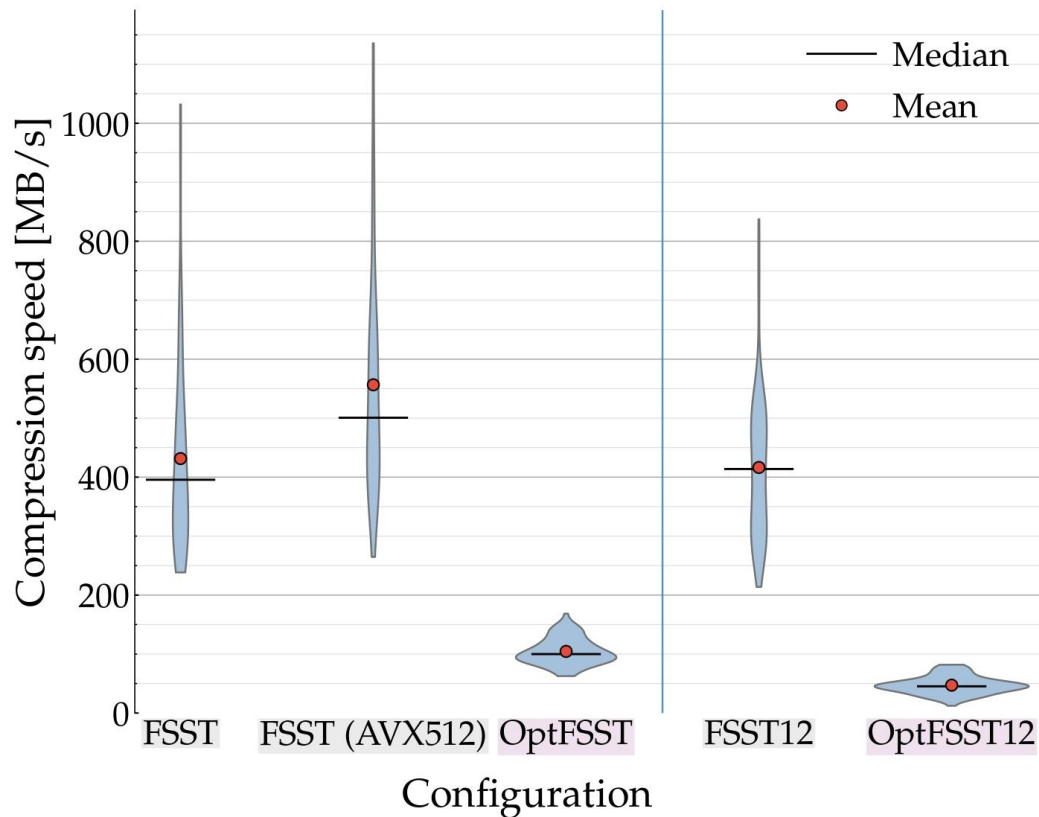
OptFSST: Compression Improvement



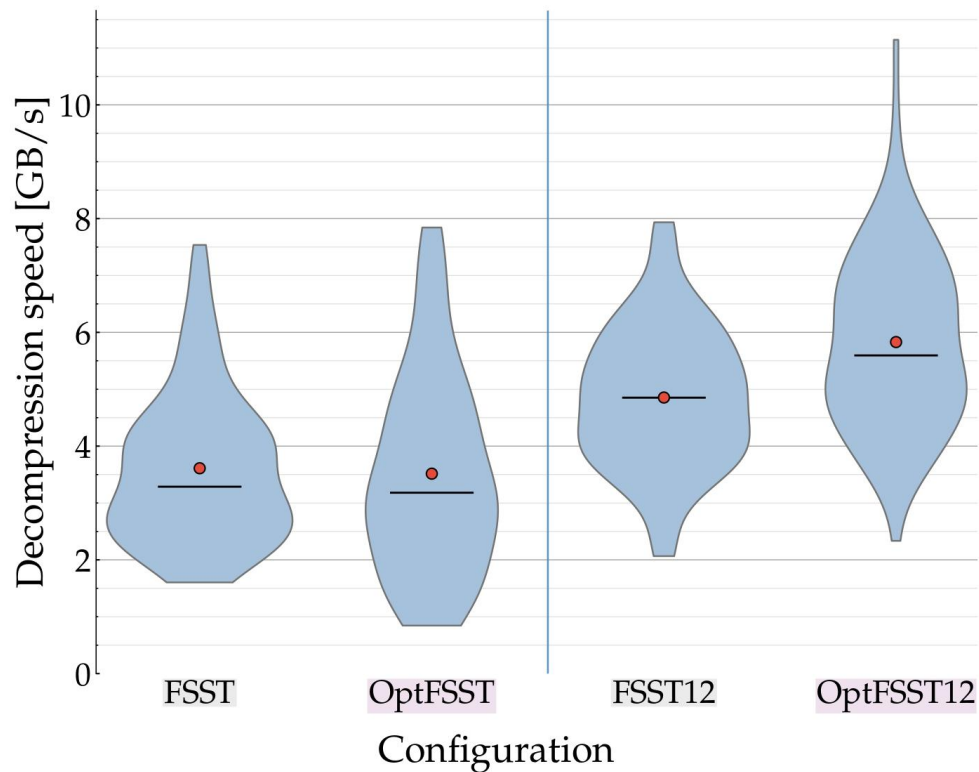
OptFSST12: Compression Improvement



OptFSST(12): Compression Speed



OptFSST(12): Decompression Speed



Summary & Future Work

- Better compression factors + decompression throughput for FSST / FSST12.
 - Avg. improvement: OptFSST: **+7.3%**, OptFSST12: **+17.0%**.
 - **1.2x** faster OptFSST12 decompression.
- NP-hardness for optimal symbol table selection.

Summary & Future Work

- Better compression factors + decompression throughput for FSST / FSST12.
 - Avg. improvement: OptFSST: **+7.3%**, OptFSST12: **+17.0%**.
 - **1.2x** faster OptFSST12 decompression.
- NP-hardness for optimal symbol table selection.
- Future work:
 1. Apply DP on the latest string encoding schemes:
 - e.g., string encoding w/ token tables (TUM paper).
 2. Run the compression on NVIDIA's Hopper architecture:
 - Comes with specialized instructions for dynamic programming.

<https://github.com/Hedi-Chehaidar/optfsst>